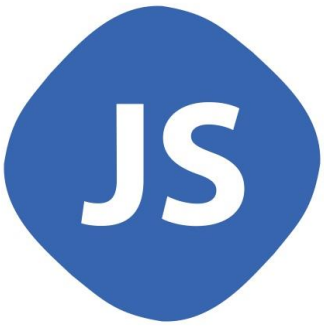


# Специальные циклы

- Разница между специальными циклами
- Цикл for ... in
- Цикл for ... of
- Заключение

## Разница между специальными циклами

Начинать изучение не со специальных циклов, а с разницы между ними возможно покажется странным решением. Но, пример показателен и наглядно демонстрирует возможности специальных циклов, поэтому поступлю именно так.

Разница между циклами **for...of** и **for...in**:

- **for...in** проходит по **именам** свойств (лучше всего подходит для перебора свойств объектов);
- **for...of** проходит по **значениям** свойств (лучше всего подходит для перебора значений массивов и HTML-коллекций).

**example_1**. Разница между специальными циклами

```
<script>
	// инициируем массив
	let arr = [3, "php", [2, 9]];
	// цикл for-in перебирает !-ключи-!
	for (let i in arr) {
		console.log(i); // 0, 1, 2
	}
```

```
    console.log("---------------");
    // цикл for-of перебирает !-значения-!
    for (let i of arr) {
        console.log(i); // 3, "php", [2, 9]
    }
</script>
```

А теперь подробности.

# Цикл for ... in

Для перебора всех свойств объекта используется цикл **for...in**. Цикл проходит по каждому отдельному свойству объекта.

**Синтаксис**

```
for (variable in object) {
    // цикл выполняется для каждого свойства объекта
}
```

**Параметры**

- **variable** – очередное имя свойства, назначается переменной на каждой итерации.
- **object** – объект, по чьим свойствам проходим в цикле.

**example_2**. Цикл for...in перебирает ключи объекта

```
<script>
    // инициируем объект
    let order = {
        id: 61,
        title: "Утепленная стеганая куртка",
```

```
        brand: "MARCO DI RADI",
        colour: "Синий",
        country: "Китай",
        hood: "Есть",
        size: "52",
        price: 25.590,
        image: "steganaya-marco.jpg"
    };
    // инициируем переменную
    out = '<ul>';
    // цикл for-in перебирает !-ключи-!
    for (key in order) {
        out += `<li>${key}: ${order[key]}</li>`;
    }
    out += '</ul>';
    // выведем в документ результат работы цикла
    document.querySelector("#content").innerHTML = out;
</script>
```

В следующем примере создадим более читаемый вывод объекта. Для этого преобразуем **значения ключей объекта из строкового типа в массив**.

**example_3**. Цикл по ключам, имеющим в качестве значений массивы

```
<script>
    let order =
    {
        id : ["Номер",1],
        title : ["Название","Утепленная стеганая куртка"],
        brand : ["Бренд","MARCO DI RADI"],
```

```
        colour : ["Цвет","Синий"],
        country : ["Страна","Китай"],
        hood : ["Капюшон","Есть"],
        size : ["Размер","52"],
        price : ["Цена",25.590],
        image: ["Путь","images/steganaya-marco.jpg"]
    };
    // инициируем переменную
    out = '<ul>';
    // цикл for-in перебирает !-ключи-!
    for (key in order) {
        out += `<li>${order[key][0]} - ${order[key][1]}</li>`;
    }
    out += '</ul>';
    // выведем в документ результат работы цикла
    document.querySelector("#content").innerHTML = out;
</script>
```

## Упорядочение свойств объекта

Упорядочены ли свойства объекта? Другими словами, если мы будем в цикле перебирать все свойства объекта, получим ли мы их в том же порядке, в котором мы их добавляли? Можем ли мы на это рассчитывать?

Свойства упорядочены особым образом: свойства с целочисленными ключами сортируются по возрастанию, остальные располагаются в порядке создания.

> ***Примечание.*** Термин **целочисленное свойство** означает строку, которая может быть преобразована в целое число и обратно без изменений.



Разберёмся подробнее. В качестве примера рассмотрим объект с автомобильными кодами регионов России в качестве ключей.

Автомобильные коды идут в порядке возрастания, потому что они являются целыми числами. Не целочисленные ключи перебираются в порядке создания.

**example_4**. Свойства объекта упорядочены

```
<script>
    // инициируем объект
    let regions = {
        "country": "Россия",
        "40": "Калужская область",
        "33": "Владимирская область",
        "23": "Краснодарский край",
        "31": "Белгородская область",
        "11": "Республика Коми",
        "explanation": "Автомобильные коды регионов",
        "note": ""
    };
    // перебираем ключи объекта
    for (key in regions) {
        document.write(key + "<p>");
    }
</script>
```

# Цикл for ... of

Цикл **for...of** появился в стандарте ES6. Предназначен для перебора **итерируемых объектов**, т.е. объектов, в которых реализован метод **Symbol.iterator**. Этот метод ещё называют **итератором**.

Именно его и использует цикл **for...of** для перебора **объектов**.

Метод **iterator** имеется у NodeList, String, Array, Map, Set и других объектов (подробнее в уроке, посвященном объектам).

**Синтаксис**

```
for (variable of iterable) {

    // цикл выполняется для каждого значения итерируемого объекта

}
```

**Параметры**

- **variable** – на каждом шаге итерации variable присваивается значение нового свойства объекта iterable. Переменная variable может быть также объявлена с помощью const, let или var.
- **iterable** – объект, перечисляемые свойства которого обходятся во время выполнения цикла.

Конструкция **for..of** помогает избежать сложностей и большого количества логики, делая код более чистым и читаемым.

## Перебор символов строки циклом for ... of

Пример использование цикла **for...of** для посимвольного перебора строки.

**example_5**. Перебор символов строки

```
<script>

    // переменная, содержащая строку

    let str = 'pechora_PRO';

    // посимвольный перебор строки

    for (let char of str) {

        console.log(char);

    }
```

```
</script>
```

## Перебор элементов массива циклом for ... of

**example_6**. Перебор элементов одномерного массива

```
<script>
    /*
    программисту придется запоминать назначения индексов
    ->
    0 - идентификатор
    1 - бренд
    2 - модель
    3 - тип ДВС
    4 - мощность л.с.
    5 - расположение цилиндров
    6 - рабочий объем
    7 - число цилиндров
    */
    // инициируем массив
    let arr = [
        '135',
        'Toyota',
        'Toyota Land Cruiser',
        'Дизельный',
        '249',
        'V-образный',
        '4461',
        '8'
```

```
    ];
    // открываем тег маркированного списка
    let html = `<ul>`;
    // for-of перебирает !-значения-!
    for (let elem of arr) {
        // значение массива в каждой итерации
        console.log("-> ", elem);
        // оборачиваем значение в теги списка
        html += `<li>${elem}</li>`;
    }
    // закрываем тег маркированного списка
    html += `</ul>`;
    // выводим список в браузер
    document.write(html);
</script>
```

**example_7**. Перебор элементов двумерного массива

```
<script>
    // инициируем массив
    // каждый вложенный массив представляет пару параметр-значение
    let arr =
    [
        ['Идентификатор','1'],
        ['Бренд', 'Toyota'],
        ['Модель', 'Toyota Land Cruiser'],
        ['Тип двигателя', 'Дизельный'],
        ['Мощность', '249'],
        ['Тип цилиндра', 'V-образный'],
```

```
        ['Объем двигателя', '4461'],
        ['Количество цилиндров', '8']
    ];
    let html = "";
    // цикл for-of перебирает !-значения-!
    for (let item of arr) {
        // на каждой итерации значением является массив
        console.log("-> ",item);
        // формируем строку для вывода
        html += `
            <p>${item[0]} - <strong>${item[1]}</strong></p>
        `;
    }
    // выводим сформированную строку в браузер
    document.write(html);
</script>
```

## Перебор элементов HTML коллекции циклом for ... of

Метод итератор у коллекции есть, а значит на этот раз переберем коллекцию HTML-элементов документа. Поработаем со списком определений.

Список определений состоит из двух элементов — **термина** и его **определения**. Сам список задается с помощью контейнера **dl**, термин — тегом **dt**, а его определение — с помощью тега **dd**.

С **версткой** списка определений поработаем достаточно подробно в следующих нескольких примерах демонстрационного кода.

**example_8**. Перебор элементов коллекции

```
<script>
```

```
    // получение коллекции определений
    var elements = document.querySelectorAll('dl > div');
    // переменная для формирования строки
    var line = "";
    // цикл for-of перебирает !-значения-!
    for (item of elements) {
        // в item попал весь блок div
        // выбор элемента dt
        dt = item.firstElementChild;
        // формируем строку для вывода
        line += `<li><a href="#">${dt.innerText}</a></li>`;
    };
    let html = `<h2>Динамический список определений</h2><ul> ${line}
    </ul>`;
    // выводим список в браузер
    document.write(html);
</script>
```

Если в предыдущем демонстрационном примере мы собрали термины списка определений и вывели их в виде списка ссылок, то теперь запишем термины в массив и выведем массив в консоль браузера.

**example_9**. Сохранение элементов коллекции в одномерный массив

```
<script>
    // получение коллекции определений
    var elements = document.querySelectorAll('dl > div');
    // переменная для формирования строки
    var line = "";
    // массив хранения элементов
```

```
    var arr = [];
    // итератор цикла
    var i = 0;
    // цикл for-of перебирает !-значения-!
    for (item of elements) {
        // в item попал весь блок div
        // выбор элемента dt
        dt = item.firstElementChild;
        // записываем элемент в массив
        arr[i] = dt.innerText;
        i++;
    };
    // вывод массива в консоль
    console.log (arr);
</script>
```

# Заключение

Даже если вы front-разработчик, вы должны четко представлять себе, что вся информация хранится в **таблицах баз данных** (почти вся, за редким исключением). Например, представленный ниже фрагмент таблицы, вполне наглядно демонстрирует хранение информации в виде табличных данных.

| id | title | brand | colour | country | size | price |
|---|---|---|---|---|---|---|
| **1** | Утепленная стеганая куртка | MARCO DI RADI | Синий | Китай | 52 | 25.590 |
| **2** | Пальто | Allsaints | Коричневый | Китай | 50 | 53.990 |
| ... | ... | ... | ... | ... | ... | ... |



Задача разработчика либо вывести данные таблицы в браузер, либо собрать данные от пользователя (из браузера) для записи их в базу данных. И в том, и в другом случае, сценарий выполняет предварительный сбор обрабатываемых данных.

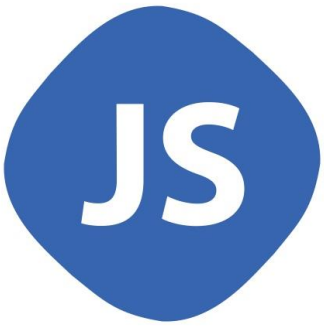
Удобно, если обрабатываемые данные будут иметь структуру, схожую с табличным представлением баз данных. Таким образом, данные в сценарии могут представляться либо в виде **массива массивов**:

```
let arr = [
    [1, "Утепленная стеганая куртка", "MARCO DI RADI", "Синий",
    "Китай", "52", "25.590"],
    [2, "Пальто", "Allsaints", "Коричневый", "Китай", "50",
    "53.990"],
    [ ... ]
];
```

Либо в виде **массива объектов**:

```
let arr2 = [
    {
        id: 1,
        title: "Утепленная стеганая куртка",
        brand: "MARCO DI RADI",
        colour: "Синий",
        country: "Китай",
        size: "52",
        price: "25.590"
    },
    {
        id: 2,
        title: "Пальто",
        brand: "Allsaints",
```





```
        colour: "Коричневый",
        country: "Китай",
        size: "50",
        price: "53.990"
    },
    { ... }
];
```

> ***Обратите внимание***, в каждом из примеров данные представлены в виде **массива**. Массив представляет собой структуру, позволяющую выполнять итерацию по своим элементам наибольшим числом способов.

Ни один из способов представления данных не имеет преимуществ, все зависит от предметной области и выполняемых задач.

> ***Примечание***. Совсем не значит, что данные представляются только рассмотренными способами. Это значит, что разобравших с предложенным ниже кодом и задачами для самостоятельного решения вы разберетесь и с другими структурами данных.

Имея сложные структуры данных, встает вопрос о наиболее эффективном способе переборе элементов этой структуры операторами циклов.

В JavaScript существуют следующие операторы цикла:

- **for** используется, когда вы заранее знаете, сколько раз вам нужно что-то сделать;
- **while** используется, когда вы не знаете, сколько раз нужно что-то сделать;
- **do ... while** работает аналогично с оператором **while**. Отличается тем, что do...while всегда выполняет выражение в фигурных скобках, по крайней мере один раз, даже если проверка условия возвращает **false**.
- **for ... of** используется для обхода **итерируемых объектов** (строк, коллекций, массивов и некоторых других);





- **for ... in** используется для обхода **свойств объектов**.

Если вы работает с **объектами** все очень просто, цикл, перебирающий свойства объекта **for...in**.

Если вы работаете со **строками**, **массивами** или **HTML-коллекциями**, вариантов немного больше:

- **for...of** – основной оператор обхода строк, массивов и HTML-коллекций, подойдет в большинстве случаев;
- **for** – если по какой-то причине не подошел первый способ или вам нужны индексы элементов. Равноправен с первым способом, используется очень часто;
- **while** – если не подошли первые два способа (что маловероятно);
- **do...while** – вряд ли будете использовать, в JS используется редко.

А теперь практикуем сохранение коллекции HTML-элементов в **двумерный массив**.

**example_10**. Сохранение данных в двумерный массив

```
<script>
    // получение коллекции определений
    var elements = document.querySelectorAll('dl > div');
    // переменная для формирования строки
    var line = "";
    // массив хранения элементов
    var arr = [];
    // итератор цикла
    var i = 0;
    // цикл for-of перебирает !-значения-!
    for (item of elements) {
        // ! не забудьте указать, что значением массива будет другой
        массив
```

```
        arr[i] = [];
        // выбор элемента dt
        dt = item.querySelector("dt");
        // выбор элемента dd
        dd = item.querySelector("dd");
        // заполняем вложенный массив
        arr[i][0] = dt.innerText;
        arr[i][1] = dd.innerText;
        i++;
    };
    // вывод массива в консоль
    console.log (arr);
</script>
```

И в **массив объектов**.

**example_11**. Сохранение данных в массив объектов

```
<script>
    // получение коллекции определений
    var elements = document.querySelectorAll('dl > div');
    // переменная для формирования строки
    var line = "";
    // массив хранения элементов
    var arr = [];
    // итератор цикла
    var i = 0;
    // цикл for-of перебирает !-значения-!
    for (item of elements) {
        // инициируем объект
```

```
        obj = {};
        // выбор элемента dt
        dt = item.querySelector("dt");
        // выбор элемента dd
        dd = item.querySelector("dd");
        // создаем свойства объекта
        obj.dt = dt.innerText;
        obj.dd = dd.innerText;
        // записываем объект в массив
        arr[i] = obj;
        i++;
    };
    // вывод массива в консоль
    console.log (arr);
</script>
```

Урок будет неполным, если не сделать обратное действие. Имея массив объектов и двумерный массив сформируем и выведем в браузер список определений.

**example_12**. Список определений из массива объектов

```
<script>
    // инициируем массив объектов
    let arr =
    [
        {dt: 'Контент', dd: 'Содержимое макета, которое не относится
        непосредственно к дизайну - тексты, графика (фотографии,
        иконки, логотипы и иллюстрации), видео.'},
        {dt: 'Композиция', dd: 'Способ размещения всех элементов
        макета друг относительно друга. Самые часто встречаемые виды
```

```
        композиции - центральная и боковая.'},
        // ...
    ]
    document.write("<dl>");
    for (item of arr) {
        document.write (
            `
            <div>
                <dt>${item.dt}</dt>
                <dd>${item.dd}</dd>
            </div>
            `
        );
    }
    document.write("</dl>");
</script>
```

**example_13**. Список определений из двумерного массива

```
<script>
    // инициируем двумерный массив
    let arr =
    [
        ['Контент', 'Содержимое макета, которое не относится
        непосредственно к дизайну - тексты, графика (фотографии,
        иконки, логотипы и иллюстрации), видео.'],
        ['Композиция', 'Способ размещения всех элементов макета друг
        относительно друга. Самые часто встречаемые виды композиции -
        центральная и боковая.'],
        // ...
```

```
    ]
    document.write("<dl>");
    for (item of arr) {
        document.write (
            `
            <div>
                <dt>${item[0]}</dt>
                <dd>${item[1]}</dd>
            </div>
            `
        );
    }
    document.write("</dl>");
</script>
```

А вот теперь все. Еще больше практики в самостоятельных работах.

В **самостоятельных работах** к уроку рассмотрим следующие вопросы:

- Загрузка данных объекта в поля формы.
- Вывод данных объекта с ключами, содержащими сложные структуры данных, в виде карточки.
- Вывод данных двумерного массива в браузер.
- Вывод данных массива объектов в браузер.
- Сохранение данных HTML-коллекции в массив объектов.